

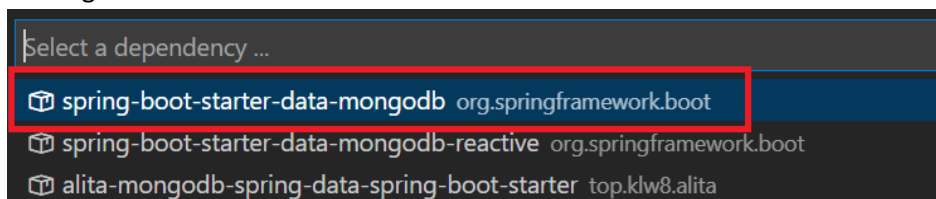
3SE2 – Übung 2

Themen: Mongo-DB Anbindung, Controller, Repository und Model, lombok, Optionals.

In dieser Übung machen wir weiter mit dem Freelancer-Jobportal. Wir wollen den Backend-Teil von der Entität «Company» umsetzen. Du kannst im bestehenden Projekt weiterarbeiten.

Verknüpfung zur MongoDB herstellen

- a) Öffne die Command Palette (Windows: Ctrl+Shift+P, Mac: ⌘⇧P) und führe den Befehl «Maven: Add a dependency..» aus. Suche nach «**spring-boot-starter-data-mongodb**». Wähle die folgende Variante:

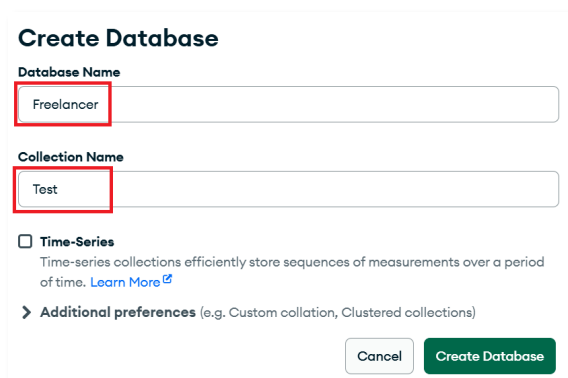


Entferne im POM-File die Version (wegen einem Problem mit SpringBoot 4).

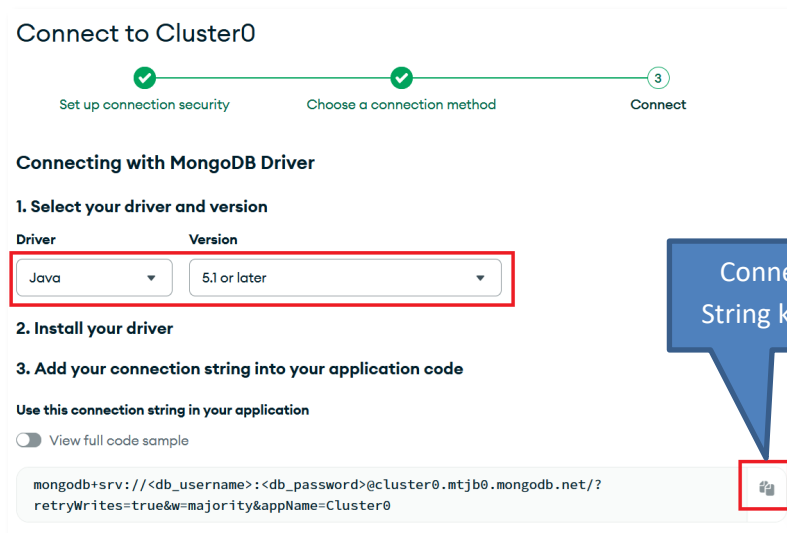
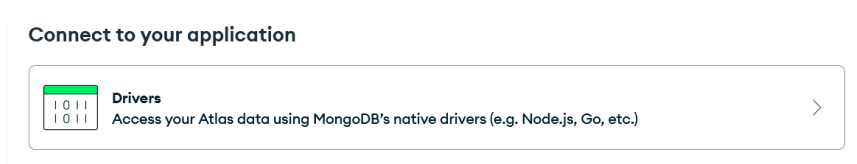
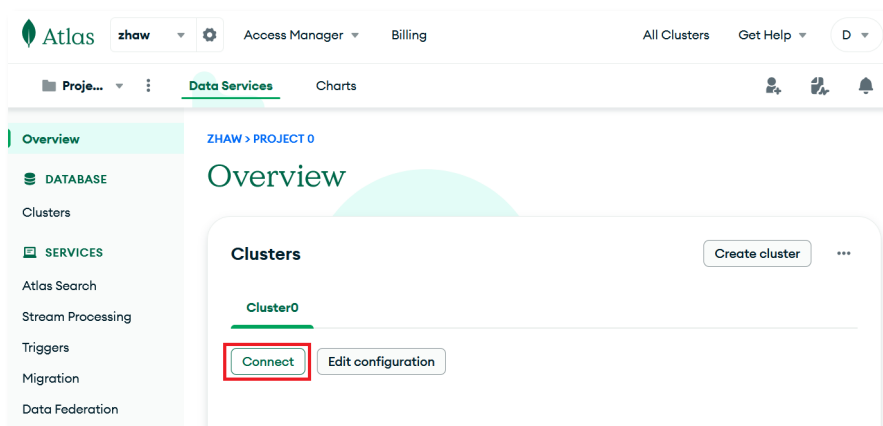
```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-mongodb</artifactId>
</dependency>
```

Danach das POM-File speichern und das Projekt synchronisieren lassen.

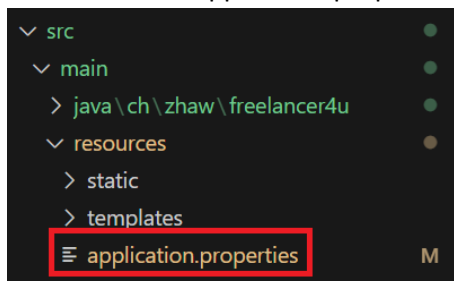
- b) Damit haben wir die nötigen Dependencies geladen. Als nächstes müssen wir die Verbindung zur MongoDB konfigurieren. Dazu benötigst du ein MongoDB-Cluster auf Atlas und die Compass-Applikation. Das hast du alles im Modul «Data Management» schon eingerichtet. Falls nötig ist im Dokument «MongoDB Setup» beschrieben, wie du dir ein MongoDB-Cluster einrichten kannst und wie Compass installiert und mit MongoDB verbunden werden kann.
- c) Erstelle mit Compass eine neue Datenbank mit dem Namen «Freelancer». Wähle für die erste Collection den Namen «Test».



- d) Logge dich unter <https://account.mongodb.com/account/login> ein und kopiere den Connection-Code wie unten gezeigt.



- e) Öffne die Datei application.properties.

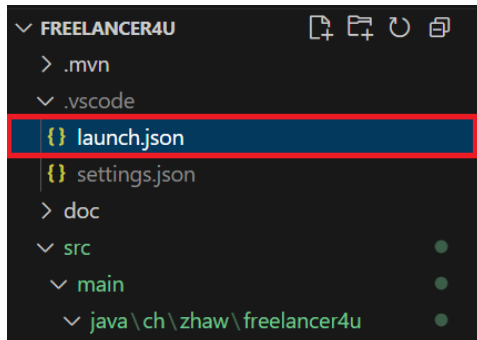


Füge die beiden folgenden Konfigurationen hinzu.

```
spring.mongodb.uri=${MONGODB_URI}
spring.mongodb.database=Freelancer
```

Hier legen wir fest, wie die verwendete DB heisst und aus welcher Umgebungsvariable der Connection-String gelesen werden kann.

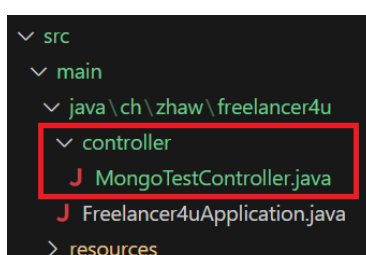
- f) Der Connection-String enthält das Passwort für den DB-Zugriff. Dieses wollen wir nicht auf GitHub speichern sondern beim Aufstarten der Applikation aus einer Umgebungsvariable lesen. Umgebungsvariablen werden beim Deployment konfiguriert (z.B. in der Azure Umgebung). Für die Entwicklungsumgebung können Umgebungsvariablen bei VS-Code in der Datei «launch.json» konfiguriert werden. Öffne die Datei:



Und ergänze sie wie unten gezeigt um den Eintrag «env» (Komma nicht vergessen). Verwende im gelb markierten Bereich den kopierten Connection-String. Ersetze <password> im Connection-String mit dem Passwort, welches du für den Admin-User angegeben hast.

```
{
  "configurations": [
    {
      "type": "java",
      "name": "Spring Boot-Freelancer4uApplication<freelancer4u>",
      "request": "launch",
      "cwd": "${workspaceFolder}",
      "mainClass": "ch.zhaw.freelancer4u.Freelancer4uApplication",
      "projectName": "freelancer4u",
      "args": "",
      "envFile": "${workspaceFolder}/.env",
      "env": {
        "MONGODB_URI": "mongodb+srv://admin..."
      }
    }
  ]
}
```

- g) Füge dem Projekt im Ordner «controller» die Klasse «MongoTestController.java» hinzu.



h) Überschreibe die Klasse mit folgendem Inhalt.

```
package ch.zhaw.freelancer4u.controller;

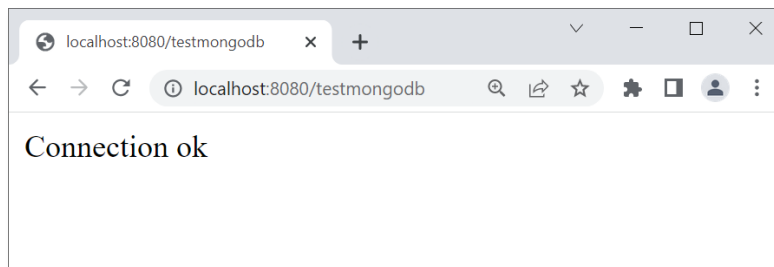
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.data.mongodb.core.MongoTemplate;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;
import com.mongodb.BasicDBObjectBuilder;
import com.mongodb.DBObject;

@RestController
public class MongoTestController {

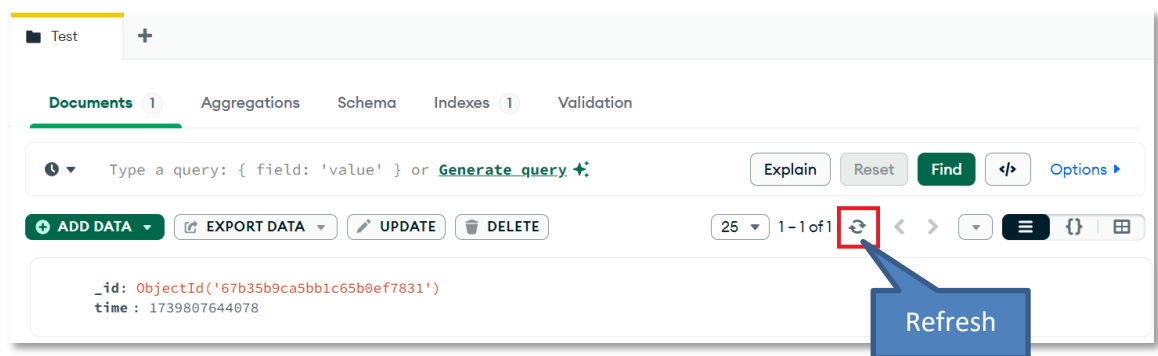
    @Autowired
    private MongoTemplate mongoTemplate;

    @GetMapping("/testmongodb")
    public ResponseEntity<String> testMongoDb() {
        String failure;
        try {
            // create test document
            Long time = System.currentTimeMillis();
            DBObject objectToSave = BasicDBObjectBuilder.start().add("time", time).get();
            // store test document
            DBObject saved = mongoTemplate.save(objectToSave, "Test");
            // retrieve test document
            DBObject read = mongoTemplate.findById(saved.get("_id"), DBObject.class, "Test");
            // verify retrieved document
            if (read != null && read.get("time").toString().equals(time.toString())) {
                return new ResponseEntity<>("Connection ok", HttpStatus.OK);
            } else {
                failure = "Retrieved document does not match";
            }
        } catch (Exception e) {
            failure = e.toString();
        }
        return new ResponseEntity<>("Failed: "+failure, HttpStatus.INTERNAL_SERVER_ERROR);
    }
}
```

- i) Stoppe und starte die Applikation neu (bei Änderungen der Umgebungsvariablen funktioniert der automatisch Restart nicht. Über dem MongoTestController kannst du jederzeit prüfen, ob die Verbindung zur MongoDB funktioniert. Beim Aufrufen von <http://localhost:8080/testmongodb> solltest du folgende Ausgabe erhalten:



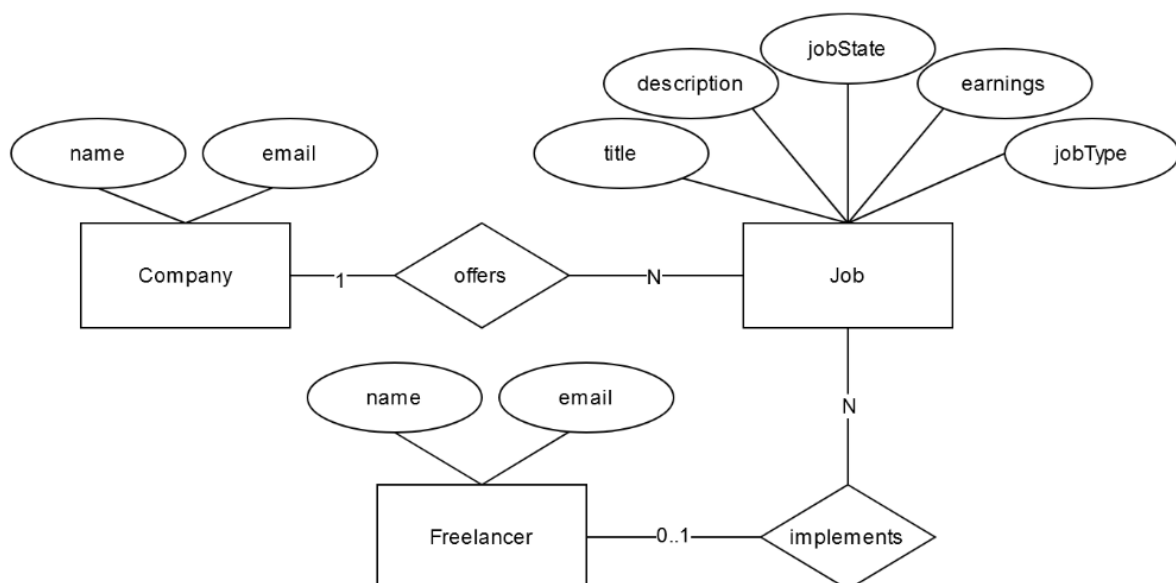
Bei jedem Aufruf des Endpoints wird in der MongoDB ein neues Dokument mit der aktuellen Systemzeit erzeugt. Prüfe, ob das Dokument gespeichert wurde.



- j) Committe die Änderungen auf GitHub (Stage -> Commit & Push). Prüfe, ob die Änderungen auf GitHub angekommen sind.

Fachliches Datenmodell für die Freelancer-App

Die folgende Abbildung definiert die fachlichen Anforderungen an das Datenmodell der Freelancer-App.



Sprint 2 planen

- a) Weiter gehts bei GitHub: Erstelle im Projekt «Freelancer4u SCRUM Planning» ein Issue mit dem Title «API endpoint add company». Die folgende Tabelle enthält die Anforderungen an den Endpoint.

POST	/api/company	Fügt eine neue Firma hinzu
Request Body	<pre>{ "name": "...", "email": "..."} </pre>	
Response Status	CREATED	Firma gespeichert
	INTERNAL_SERVER_ERROR	Speichern fehlgeschlagen
Response Body	<pre>{ "id": "...", "name": "...", "email": "..."} </pre>	

Die Beschreibung des Issues soll die Anforderungen aus der Tabelle enthalten und beschreiben, wie die Anforderung geprüft werden kann. Auf der nächsten Seite ist eine mögliche Lösung abgebildet.

Create new issue

Add a title *

API endpoint add company

Add a description

Write Preview

Requirements

Add the endpoint to add new companies.

Item	Description
Method	POST
URL	/api/company
Request body	<pre>{"name": "...", "email": "..."} </pre>
Response status	CREATED : company added INTERNAL_SERVER_ERROR : adding failed
Response body	<pre>{"id": "...", "name": "...", "email": "..."} </pre>

Test

- ☐ POST request with valid request body returns **CREATED** and a response body with id. Company is stored in DB.
- ☐ POST request with invalid request body returns **INTERNAL_SERVER_ERROR** and an empty response body. No modification in DB.

Verwende das vorhandene Label «enhancement», und das neu erstellte Label «backend». Weise das Issue dir zu und wähle unter Projects das zugehörige Projekt.

Description zum Kopieren:

```
**Requirements**
Add the endpoint to add new companies.
| Item| Description|
| :----- | :----- |
| Method | `POST` |
| URL | `/api/company` |
| Request body | `{"name": "...", "email": "...}"` |
| Response status | `CREATED`: company added<br/> `INTERNAL_SERVER_ERROR`: adding failed |
| Response body | `{"id": "...", "name": "...", "email": "...}"` |

**Test**
- [ ] POST request with valid request body returns `CREATED` and a response body with id. Company is stored in DB.
- [ ] POST request with invalid request body returns `INTERNAL_SERVER_ERROR` and an empty response body. No modification in DB.
```

- b) Erstelle zwei zusätzliche Issues für die folgenden Endpoints. Verwende für beide Issues die Label «enhancement» und «backend» und weise sie dir und dem Projekt zu.

GET	/api/company	Gibt die Liste aller Firmen zurück
Responses	OK	[{ "id": "6344ef34efc7c92e1ad8412", "name": "ENCOM", "email": "contact@encom.com" }, {...}]
Bemerkung	Sind keine Firmen vorhanden, wird OK und ein leeres Array zurückgegeben.	

GET	/api/company/{id}	Gibt die Firma mit der angegebenen ID zurück
Responses	OK	{ "id": "6344e458927b2b7d84923680", "name": "ENCOM", "email": "contact@encom.com" }
	NOT_FOUND	Company nicht gefunden

- c) Setze alle Issues auf Ready und plane sie für «Sprint 2».

Freelancer4u SCRUM Planning

Current iteration | Next iteration | Prioritized backlog | Roadmap | In review | My items | + New view

Search: assignee:@me

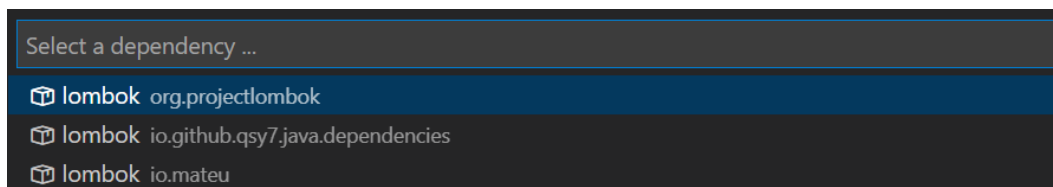
Status	Title	Labels	Status	Iteration	Assignees
Ready This is ready to be picked up 3	1 Use Case Diagram #1	documentation	Done	Iteration 1	DavidZhaw
Done This has been completed 1	2 API endpoint ad... #2	backend enhancement	Ready	Iteration 2	DavidZhaw
	3 API endpoint re... #3	backend enhancement	Ready	Iteration 2	DavidZhaw
	4 API endpoint re... #4	backend enhancement	Ready	Iteration 2	DavidZhaw

Prüfe unter «Current iteration» ob die Issues korrekt im SCRUM-Board erscheinen.

Bemerkung: Falls du die Übung vorzeitig löst, sind die Issues in «Next Iteration».

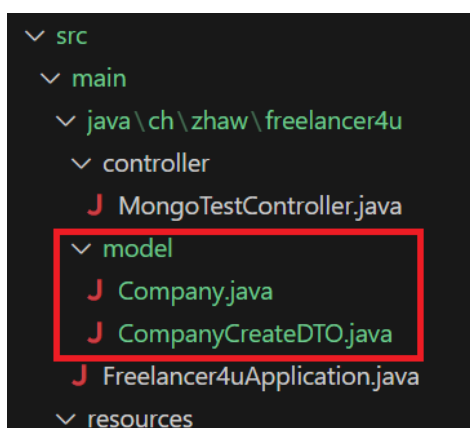
Issue #2 implementieren

- a) Schiebe das Issue #2 (API endpoint add company) im Project-Board in die «In Progress» Spalte.
- b) Für die Datenklassen wollen wir lombok verwenden. Öffne die Command Palette und führe den Befehl «Maven: Add a dependency...» aus. Suche nach «lombok» und wähle folgende Variante:



Speichere das POM-File und synchronisiere den classpath.

- c) Für die Company benötigen wir zwei Datenklasse. Die Klasse CompanyCreatedDTO definiert die Struktur der Daten, die zum Erstellen einer neuen Firma benötigt werden. Die Klasse Company enthält dagegen alle Daten, die in der Datenbank gespeichert werden sollen. Diese wird auch als DAO bezeichnet. Füge die neuen Klassen im Ordner «model» hinzu.



- d) Gemäss dem Datenmodell von Seite 5 hat jede Company die Attribute name und email. Wir erstellen die folgende DTO Klasse um Companies zu erstellen. Da das DTO direkt über den Endpoint entgegengenommen wird, genügt ein parameterloser Konstruktor:

```
package ch.zhaw.freelancer4u.model;

import lombok.Getter;
import lombok.NoArgsConstructor;

@NoArgsConstructor
@Getter
public class CompanyCreatedDTO {
    private String name;
    private String email;
}
```

- e) Die DAO Klasse für Company enthält zusätzlich die id. Weil wir diese Klasse mit den Daten aus dem DTO instanziiieren wollen, braucht es einen Konstruktor, über den das E-Mail und der Name gesetzt werden können. Mit @NonNull und dem @RequiredArgsConstructor kann ein solcher erstellt werden. Durch die Annotation @Document("company") wird das Objekt der MongoDB-Collection «company» zugeordnet. Diese wird beim ersten Schreibzugriff automatisch erstellt.

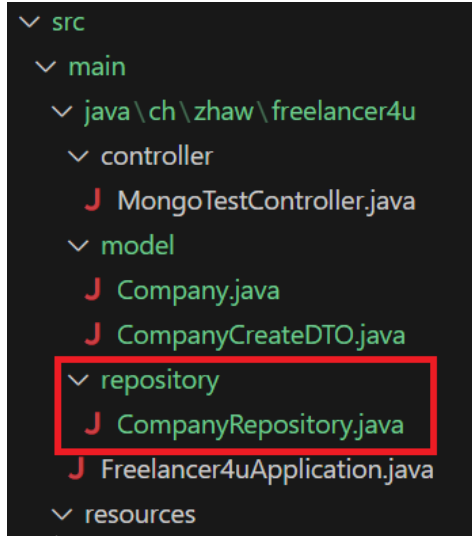
```
package ch.zhaw.freelancer4u.model;

import org.springframework.data.annotation.Id;
import org.springframework.data.mongodb.core.mapping.Document;

import lombok.NonNull;
import lombok.RequiredArgsConstructor;
import lombok.Getter;
import lombok.NoArgsConstructor;

@NoArgsConstructor
@RequiredArgsConstructor
@Getter
@Document("company")
public class Company {
    @Id
    private String id;
    @NonNull
    private String name;
    @NonNull
    private String email;
}
```

- f) Erstelle als nächstes das Repository. Beachte den Ordner:



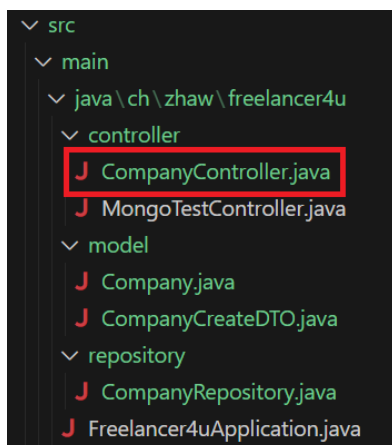
- g) Ergänze die Klasse wie unten gezeigt. Durch das Erben von MongoRepository erhalten wir alle Basis-Methoden, um auf die Collection zuzugreifen.

```
package ch.zhaw.freelancer4u.repository;

import org.springframework.data.mongodb.repository.MongoRepository;
import ch.zhaw.freelancer4u.model.Company;

public interface CompanyRepository extends MongoRepository<Company,String>{
}
```

- h) Nun fehlt noch der Controller mit den Endpoints. Füge die Datei CompanyController.java im Ordner controller hinzu.



- i) Der folgende Programmcode enthält einen ersten Endpoint für Post-Requests. Mit der Annotation `@RequestMapping` kann ein gemeinsamer Pfad angegeben werden. Alle Endpoints in diesem Controller beginnen daher mit `/api`. Der vollständige Pfad für den Company Endpoint ist damit `/api/company`. Die Daten treffen im `CompanyCreateDTO`-Format ein. Mit diesen Angaben wird ein `Company` Objekt instanziiert und in die Datenbank geschrieben.

```
package ch.zhaw.freelancer4u.controller;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RestController;
import org.springframework.web.bind.annotation.RequestMapping;

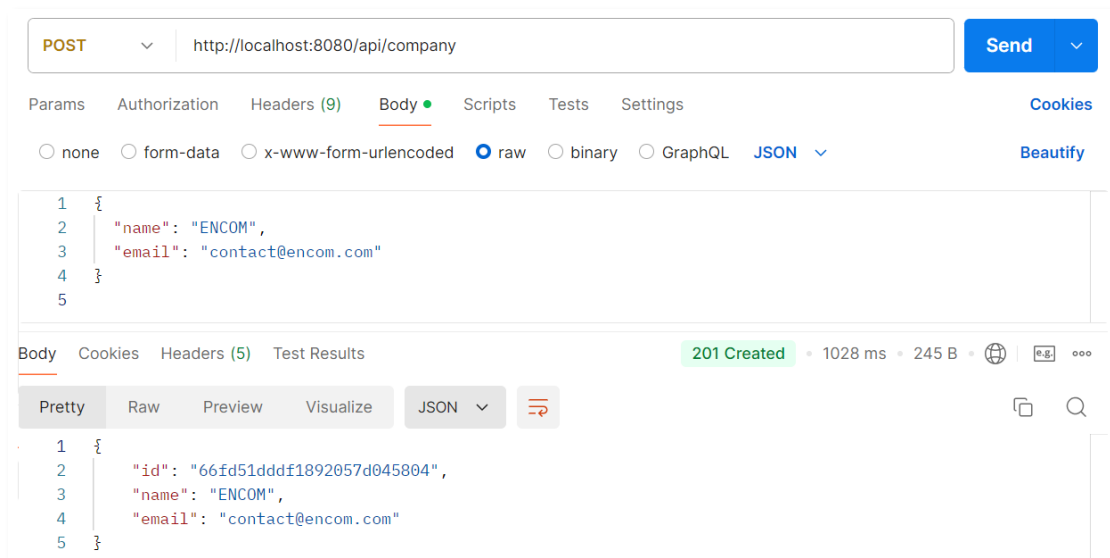
import ch.zhaw.freelancer4u.model.Company;
import ch.zhaw.freelancer4u.model.CompanyCreateDTO;
import ch.zhaw.freelancer4u.repository.CompanyRepository;

@RestController
@RequestMapping("/api")
public class CompanyController {

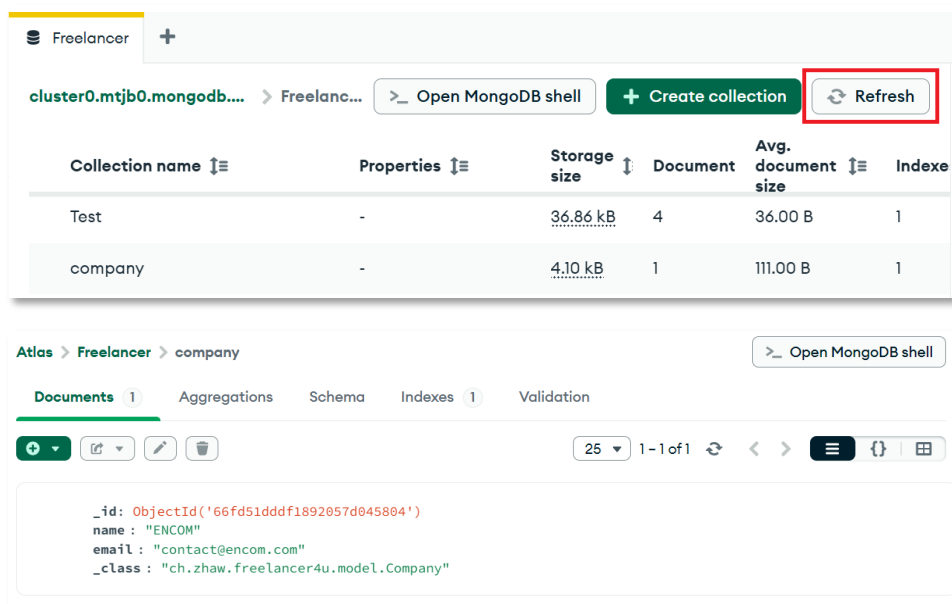
    @Autowired
    CompanyRepository companyRepository;

    @PostMapping("/company")
    public ResponseEntity<Company> createCompany(
        @RequestBody CompanyCreateDTO fDTO) {
        Company fDAO = new Company(fDTO.getName(), fDTO.getEmail());
        Company f = companyRepository.save(fDAO);
        return new ResponseEntity<>(f, HttpStatus.CREATED);
    }
}
```

- j) Um den Endpoint zu testen, benötigen wir Postman. Falls du Postman noch nicht hast, installiere diesen. Teste den Post-Request wie unten gezeigt.



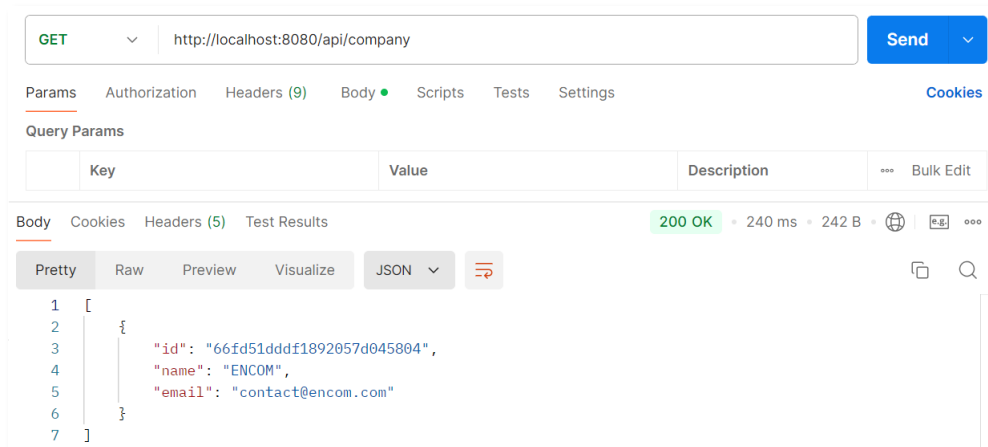
- k) Prüfe mit Compass, ob die Collection erstellt und ob das Company-Dokument korrekt gespeichert wurde. Du musst bei der DB einen refresh machen, damit die neue Collection angezeigt wird.



- l) Teste das Verhalten für einen ungültigen Request Body (INTERNAL_SERVER_ERROR)
 m) Committe die Änderungen mit der Message «Implemented post endpoint for company. Closes #2.». Prüfe ob Issue #2 nun im SCRUM Board in der Spalte «Done» erscheint.

Issue #3 implementieren

- a) Schiebe das Issue #3 (API endpoint read all companies) im Project-Board in die «In Progress» Spalte.
 b) Implementiere das Issue gemäss den Anforderungen. Verwende die Methode `findAll` aus dem Repository und eine `ResponseEntity` als Rückgabewert.
 c) Teste auch diesen Endpoint mit Postman. Es ist sinnvoll, wenn du alle Request zum Freelancer in einer Collection gruppierst. Wenn alles funktioniert, committe deine Änderungen schliesse das Issue mit der passenden Commit Message



Issue #4 implementieren

- a) Schiebe das Issue #4 (API endpoint read company by id) im Project-Board in die «In Progress» Spalte.
- b) Implementiere das Issue gemäss den Anforderungen. Du benötigst eine PathVariable um die id in der URL zu erhalten. Verwende die von MongoRepository geerbte Methode findById zum Auslesen der Company. Diese Methode gibt ein Optional zurück. Achte darauf, dass du die richtige Library (java.util.Optional und nicht die von Apache). Falls die Company gefunden wurde, musst du diese aus dem Optional extrahieren und zurückgeben. Falls nicht, musst du eine Response mit dem Status-Code NOT_FOUND zurückgeben.
- c) Prüfe mit Postman, ob bei einer gültigen ID das Objekt und bei einer ungültigen ID der Status-Code NOT_FOUND zurückkommt. Committe die Änderung und schliesse das Issue mit der passenden Commit Message.
- d) Gib zum Schluss die URL deines Repos erneut auf Moodle ab. Great Job 😊 !